

CRM Sync — Event-Driven Integration Spec

Version 1.0 · 2026-05-27 · Specification. Cron is the wrong primitive for data integration — most polls find nothing and the hits arrive up to fifteen minutes stale. This spec moves the external integrations to an event-driven substrate: **data moves when something happens, not when a clock ticks**, with ordering, replay, and observability defined per channel, and cron demoted to the reconcile backstop.

Version: 1.0 Date: 2026-05-27 Status: Specification

Replaces: Cron-only polling for external integrations

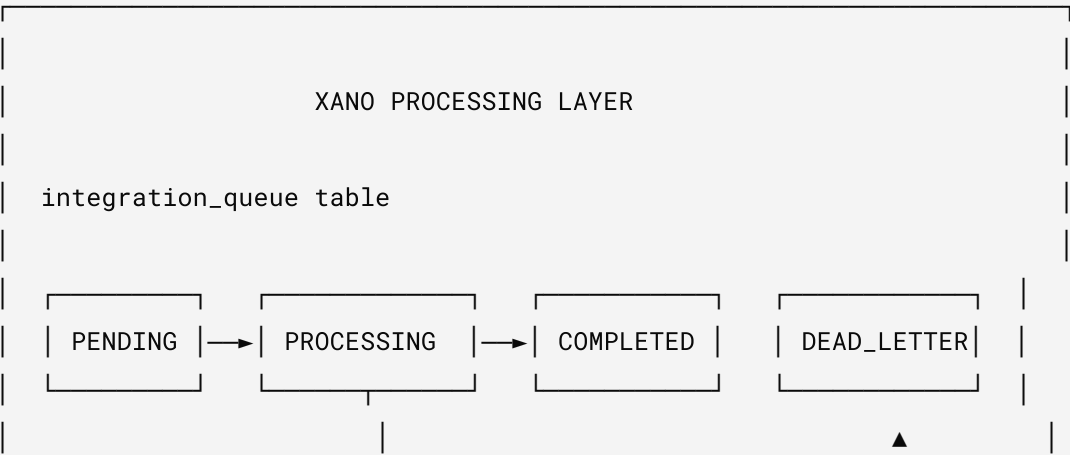
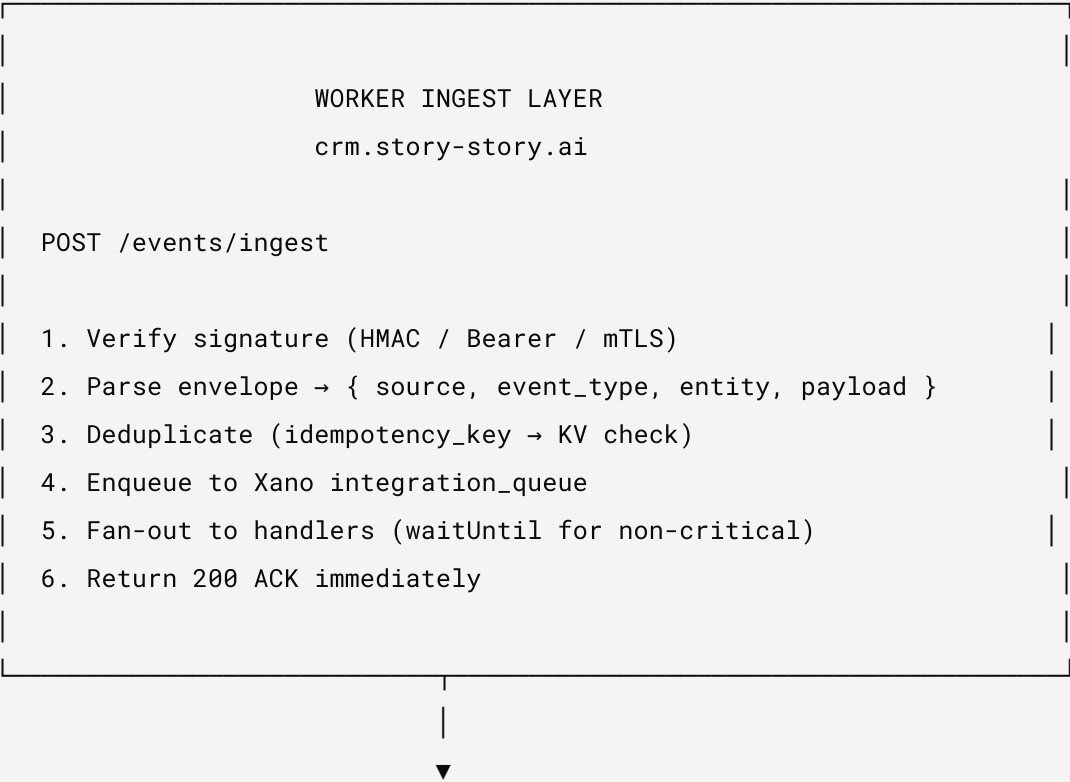
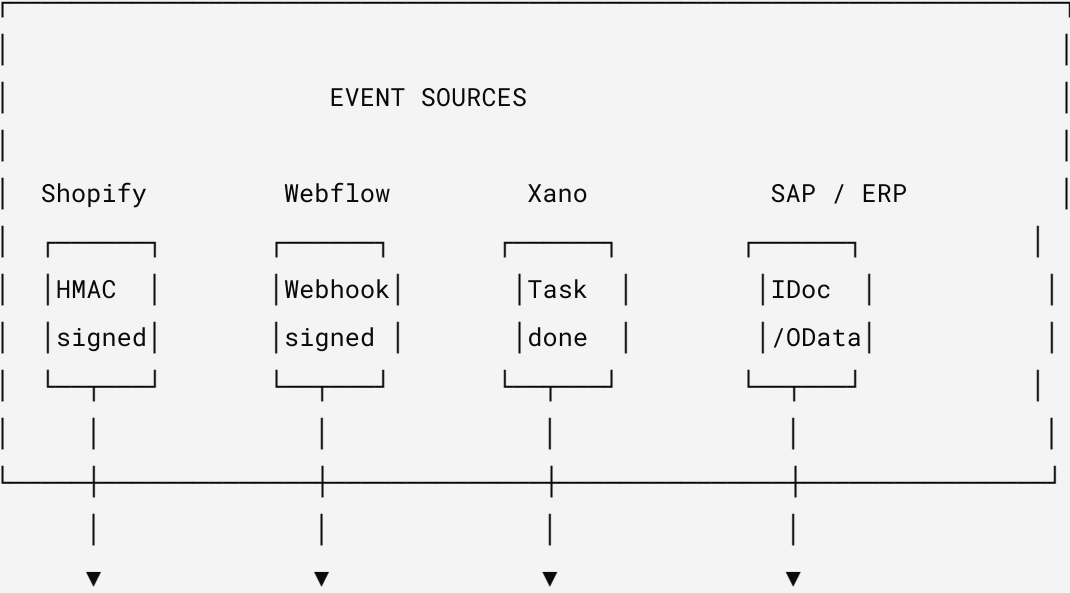
1. PROBLEM WITH CRON POLLING

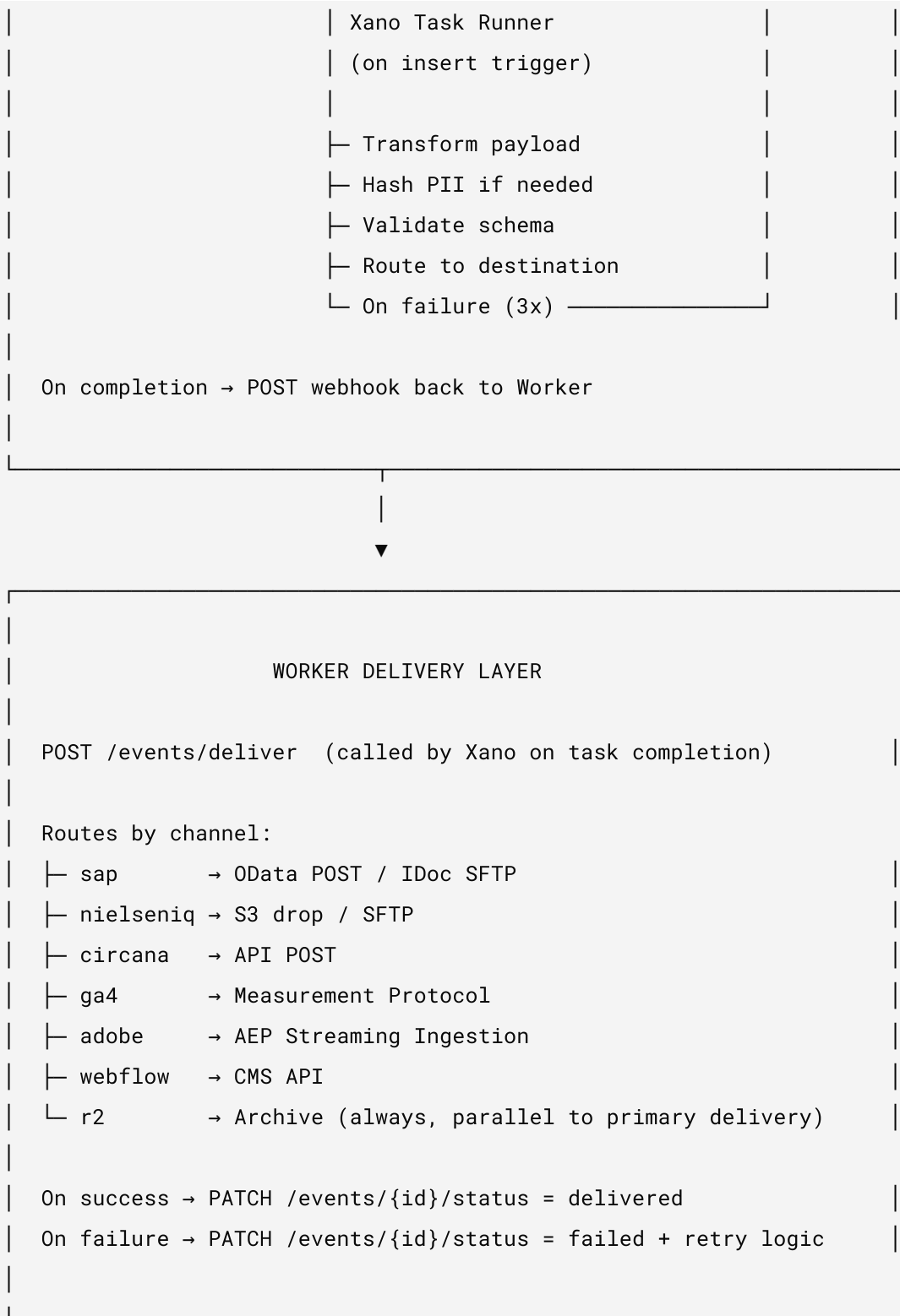
```
Cron (* / 15 * * * *)
|
├─ 95% of runs: no new data → wasted compute
├─ 5% of runs: data ready → up to 15 min stale
├─ No ordering guarantee across runs
└─ Scales poorly: more channels = more polls per tick
```

Cron is the wrong primitive for data integration. It was designed for periodic maintenance (token refresh, customer sync), not for reactive data flows.

2. EVENT-DRIVEN ARCHITECTURE

Principle: Data moves when something happens, not when a clock ticks.





3. EVENT ENVELOPE SCHEMA

Every event entering the system uses a standard envelope, regardless of source:

```

interface EventEnvelope {
  // Identity
  event_id: string;           // UUID – set by source or generated on ingest
  idempotency_key: string;    // SHA-256(source + entity_id + event_type + timestamp)

  // Routing
  source: EventSource;        // "shopify" | "webflow" | "xano" | "sap" | "erp" | "manual"
  event_type: string;         // "customer.updated" | "order.created" | "inventory.changed"
  entity_type: EntityType;    // "customer" | "order" | "product" | "inventory" | "invoice"
  entity_id: string;          // Source-system ID

  // Scoping
  tenant_shop: string;        // Multi-tenant isolation

  // Payload
  payload: Record<string, unknown>; // Source-specific data

  // Metadata
  timestamp: string;          // ISO 8601 – when the event occurred (not when received)
  received_at: string;        // When worker ingested it

  // Delivery
  destinations: string[];     // ["sap", "ga4", "r2"] – fan-out targets
  priority: 1 | 2 | 3 | 4 | 5; // 1=critical (inventory), 5=low (analytics batch)
}

```

4. WORKER: EVENT INGEST ENDPOINT

POST /events/ingest

Single entry point for all external events. Replaces per-source webhook endpoints over time.

```

async function handleEventIngest(
  cfg: ResolvedConfig,
  env: Env,
  request: Request,
  origin: string | null,
  rotatableKey: string | null
): Promise<Response> {

  // 1. Authenticate - support multiple auth methods
  const authResult = await authenticateEventSource(cfg, env, request, rotatableKey);
  if (!authResult.authenticated) {
    return jsonResponse({ error: "Unauthorized", method: authResult.method }, 401, origin);
  }

  // 2. Parse envelope
  const raw = await request.json() as Partial<EventEnvelope>;
  const envelope = normalizeEnvelope(raw, authResult.source);

  // 3. Idempotency check - reject duplicates
  const dedupeKey = `evt:${envelope.idempotency_key}`;
  const existing = await env.CRM_STATE.get(dedupeKey);
  if (existing) {
    return jsonResponse({ ok: true, deduplicated: true, event_id: envelope.event_id }, 200, ori
  }

  // 4. Store idempotency marker (TTL: 7 days)
  await env.CRM_STATE.put(dedupeKey, envelope.event_id, { expirationTtl: 604800 });

  // 5. Enqueue to Xano for processing
  const queued = await enqueueToXano(cfg, envelope);

  // 6. Fire-and-forget side effects (non-blocking)
  //   These run after the 200 response via waitUntil pattern
  const ctx = { waitUntil: (p: Promise<unknown>) => p }; // simplified

  // Immediate fan-out for low-latency destinations
  if (envelope.destinations.includes("ga4") && envelope.priority <= 2) {
    // GA4 gets real-time push for high-priority events
    ctx.waitUntil(pushEventToGA4(cfg, envelope));
  }
}

```

```
if (envelope.destinations.includes("r2")) {  
  // Always archive  
  ctx.waitUntil(archiveToR2(env, envelope));  
}  
  
// 7. ACK immediately - Xano handles the rest  
return jsonResponse({  
  ok: true,  
  event_id: envelope.event_id,  
  queued: queued,  
  destinations: envelope.destinations,  
}, 202, origin); // 202 Accepted - processing async  
}
```

Authentication per source

```
async function authenticateEventSource(
  cfg: ResolvedConfig,
  env: Env,
  request: Request,
  rotatableKey: string | null
): Promise<{ authenticated: boolean; source: string; method: string }> {

  const url = new URL(request.url);
  const sourceHint = url.searchParams.get("source") || request.headers.get("x-event-source") ||

  // Shopify HMAC
  if (sourceHint === "shopify" || request.headers.has("x-shopify-hmac-sha256")) {
    const { valid } = await verifyShopifyHmac(request, cfg.shopifyAppSecret);
    return { authenticated: valid, source: "shopify", method: "hmac" };
  }

  // SAP client certificate or OAuth token
  if (sourceHint === "sap") {
    const token = request.headers.get("authorization")?.replace("Bearer ", "");
    if (token) {
      const valid = await verifySAPToken(cfg, token);
      return { authenticated: valid, source: "sap", method: "oauth2" };
    }
  }

  // Xano task callback (shared secret)
  if (sourceHint === "xano" || request.headers.has("x-xano-task-id")) {
    const secret = request.headers.get("x-xano-webhook-secret") || "";
    const valid = secret === cfg.xanoWebhookSecret;
    return { authenticated: valid, source: "xano", method: "shared_secret" };
  }

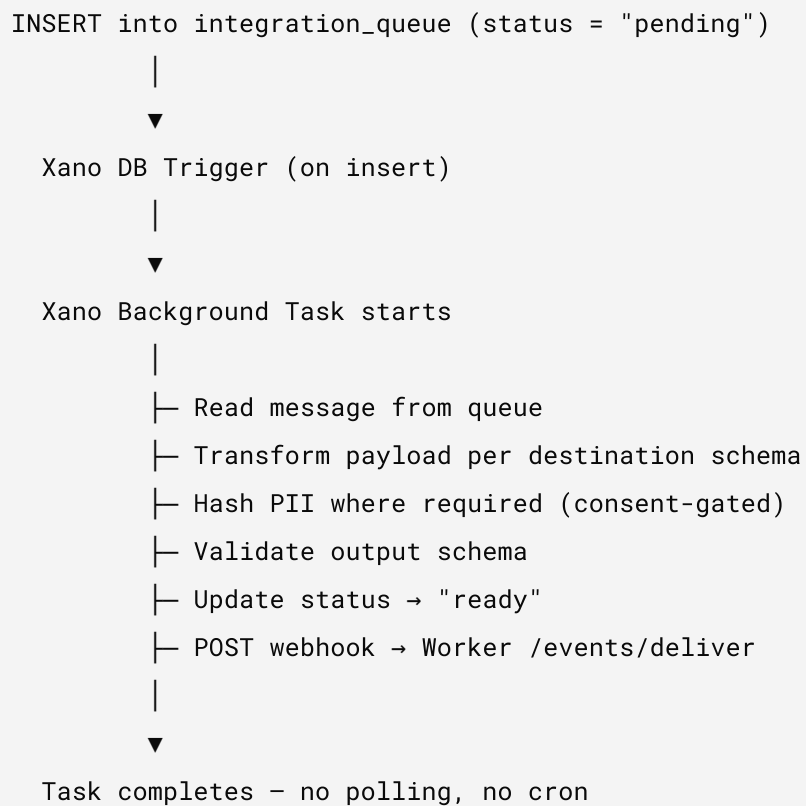
  // Admin key / rotatable key (manual, ERP generic, testing)
  if (verifyBearerToken(cfg as unknown as Env, request, undefined, rotatableKey)) {
    return { authenticated: true, source: sourceHint || "admin", method: "admin_key" };
  }
}
```

```
return { authenticated: false, source: "unknown", method: "none" };  
}
```

5. XANO: TASK RUNNER (NOT CRON)

How Xano Task Runner works

Xano's task system is **event-triggered**, not time-based:



Xano API Endpoint: `POST /api:{group}/integration-enqueue`

Called by Worker to insert events into the queue:

POST /api:{group}/integration-enqueue

Authorization: Bearer {XANO_API_KEY}

Content-Type: application/json

```
{
  "event_id": "evt_a1b2c3d4",
  "source": "shopify",
  "event_type": "customer.updated",
  "entity_type": "customer",
  "entity_id": "gid://shopify/Customer/12345",
  "tenant_shop": "hx-stage.myshopify.com",
  "payload": { ... },
  "destinations": ["sap", "ga4", "r2"],
  "priority": 2,
  "timestamp": "2026-05-27T14:30:00Z"
}
```

Response: { "ok": true, "queue_id": 4521 }

Xano DB Trigger: `on_queue_insert`

TRIGGER: AFTER INSERT on integration_queue

CONDITION: NEW.status = 'pending'

ACTION: Start background task "process_integration_event"

INPUT: NEW.id

Xano Background Task: `process_integration_event`

INPUT: `queue_id` (int)

1. FETCH queue row by id
2. SET status = "processing"
3. SWITCH on destination:

CASE "sap":

- Map customer fields → SAP Business Partner schema
- Map order fields → SAP Sales Order schema
- Validate required SAP fields (`bukrs`, `vkorg`, `vtweg`)
- Format as OData JSON or IDoc XML

CASE "nielseniq":

- Filter: check `consent_records` for `analytics_partners` scope
- Hash email: `SHA-256(lowercase(email))`
- Map Shopify SKU → UPC via `product_upc_mapping` table
- Format as NielsenIQ CSV row

CASE "circana":

- Same consent + hash as `nielseniq`
- Format as Circana JSON schema

CASE "erp_generic":

- Apply tenant-specific field mapping from `integration_mappings` table
- Validate against destination schema

4. SET status = "ready", `transformed_payload` = result

5. POST webhook to Worker:

POST `{worker_url}/events/deliver`

Authorization: Bearer `{WORKER_WEBHOOK_SECRET}`

Body: {

```
"queue_id": queue_id,
"event_id": original_event_id,
"channel": destination,
"transformed_payload": result,
"delivery_config": { method, endpoint, credentials_key }
```

```
}
```

```
6. On webhook 2xx → SET status = "delivered"
```

```
On webhook fail → INCREMENT retry_count
```

```
IF retry_count >= max_retries → SET status = "dead_letter"
```

```
ELSE → SET status = "pending" (re-triggers task)
```

6. WORKER: DELIVERY ENDPOINT

POST /events/deliver

Called by Xano when a message is processed and ready for external delivery:

```

async function handleEventDeliver(
  cfg: ResolvedConfig,
  env: Env,
  request: Request,
  origin: string | null
): Promise<Response> {

  // Auth: Xano webhook secret
  const secret = request.headers.get("x-xano-webhook-secret") || "";
  if (secret !== cfg.xanoWebhookSecret) {
    return jsonResponse({ error: "Unauthorized" }, 401, origin);
  }

  const msg = await request.json() as DeliveryMessage;

  // Always archive to R2 (parallel, non-blocking)
  const archivePromise = env.ANALYTICS_EXPORTS.put(
    `events/${msg.channel}/${new Date().toISOString().slice(0,10)}/${msg.event_id}.json`,
    JSON.stringify(msg),
  );

  // Route to delivery handler
  let result: DeliveryResult;
  switch (msg.channel) {
    case "sap":
      result = await deliverToSAP(cfg, env, msg);
      break;
    case "nielseniq":
      result = await deliverToAnalyticsPlatform(cfg, env, msg, "sftp");
      break;
    case "circana":
      result = await deliverToAnalyticsPlatform(cfg, env, msg, "api");
      break;
    case "ga4":
      result = await deliverToGA4Direct(cfg, msg);
      break;
    case "adobe":
      result = await deliverToAdobeAEP(cfg, env, msg);
      break;
    case "webflow":

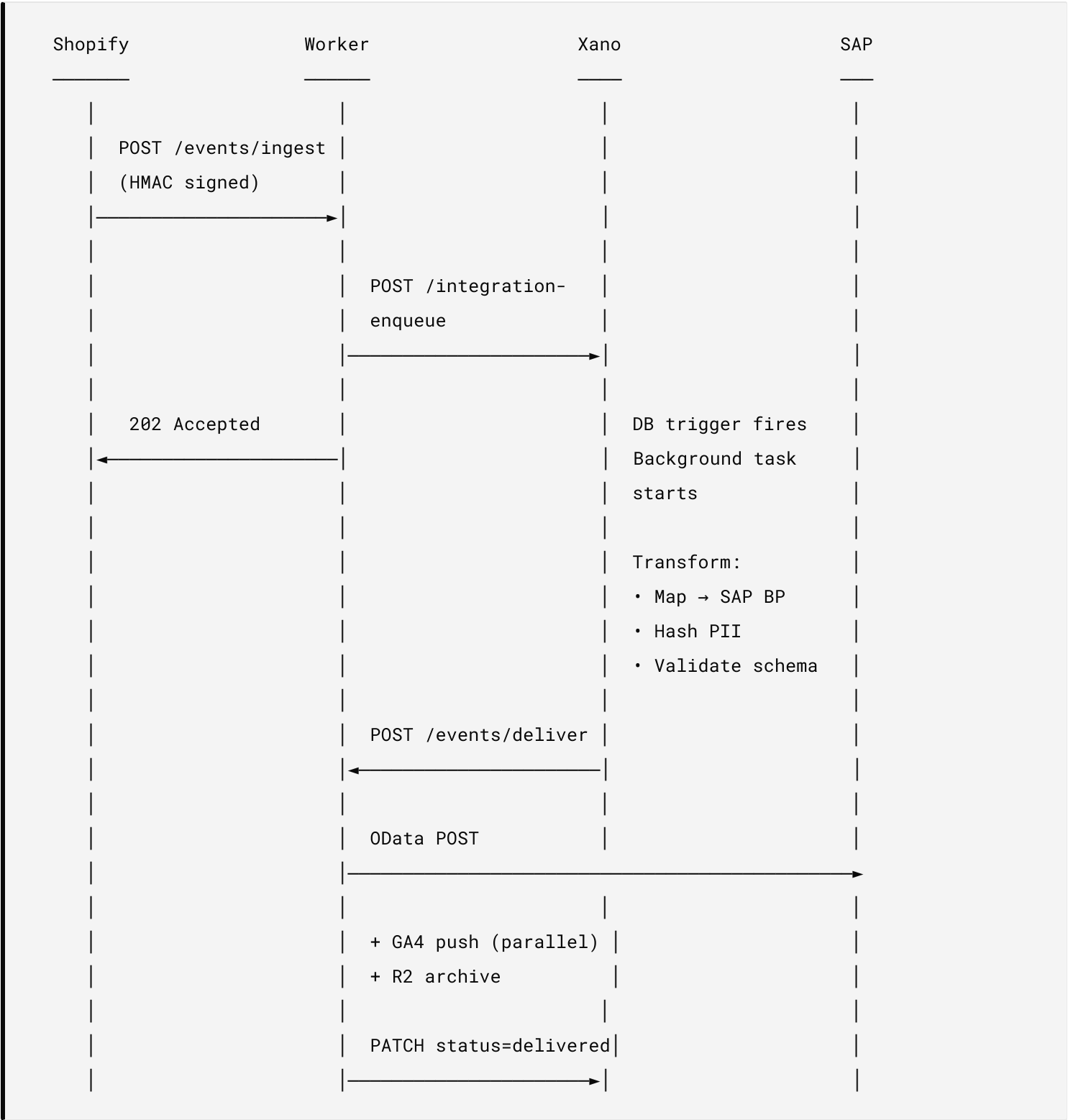
```

```
        result = await deliverToWebflowCMS(cfg, msg);
        break;
    default:
        result = { ok: true, log: { archived: true } };
    }

    await archivePromise; // ensure archive completes

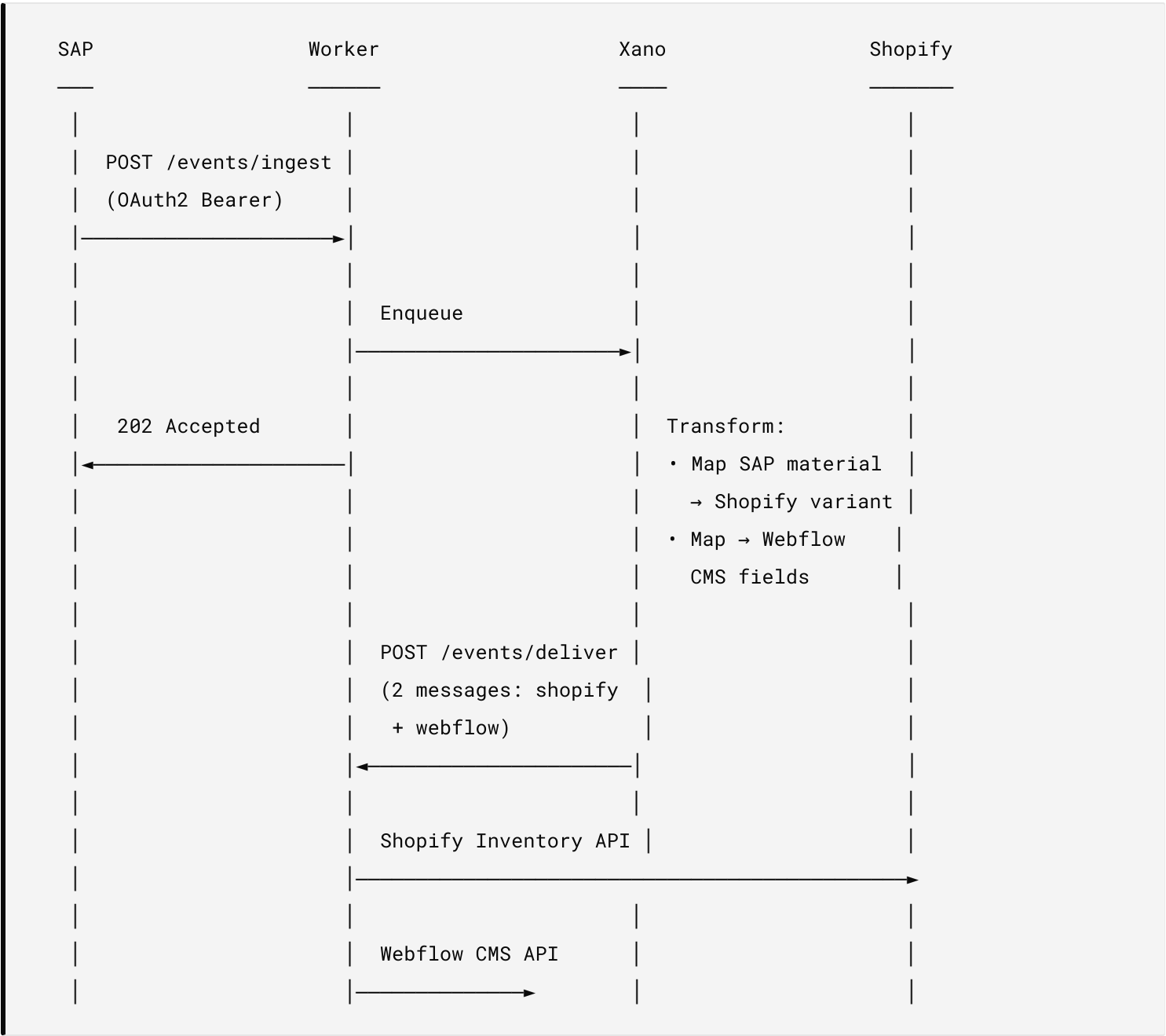
    return jsonResponse({
        ok: result.ok,
        event_id: msg.event_id,
        channel: msg.channel,
        delivery_log: result.log,
    }, result.ok ? 200 : 502, origin);
}
```

7.1 Shopify Customer Updated → SAP + GA4 + R2

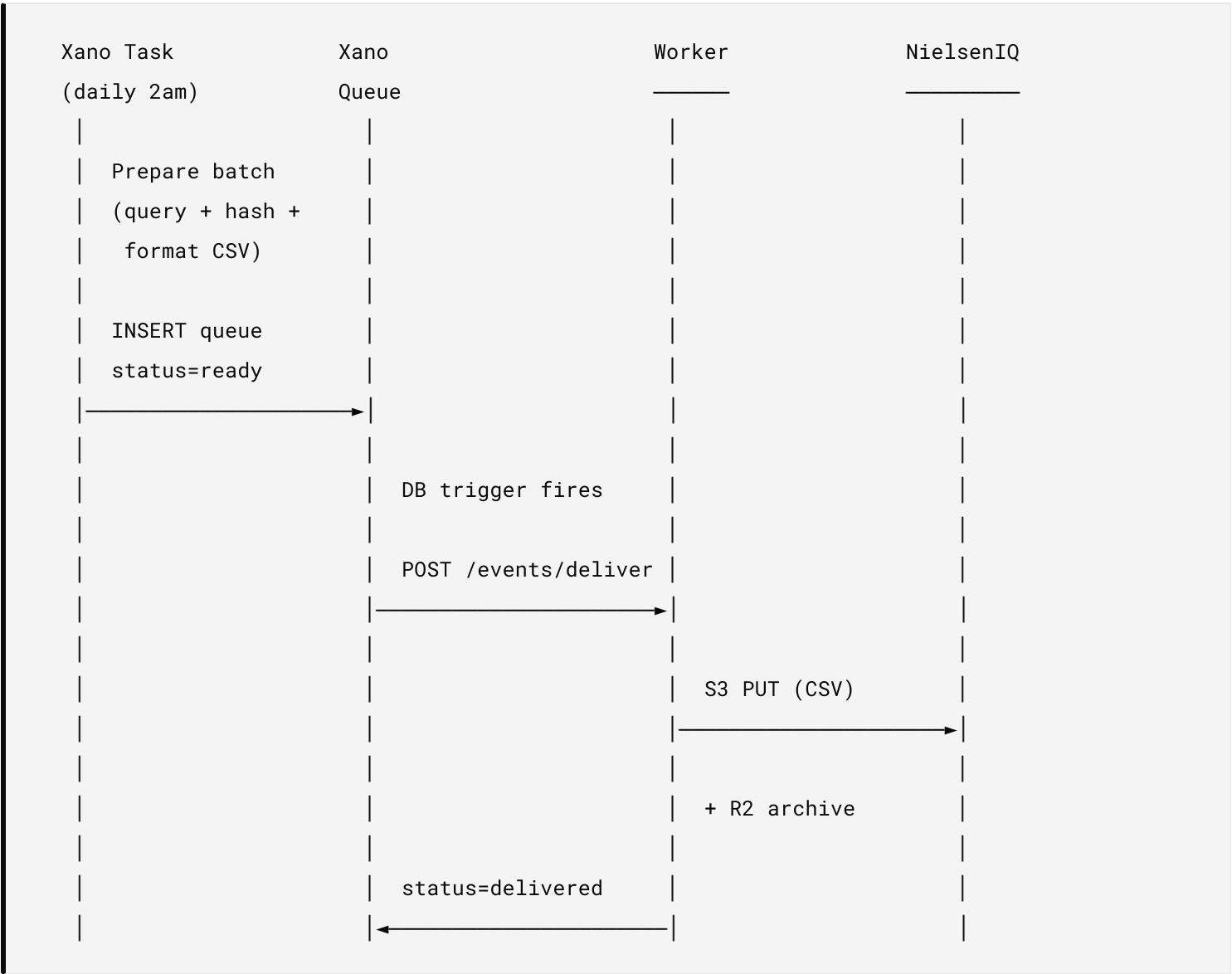


Total latency: 2-5 seconds (vs 15 min with cron polling)

7.2 SAP Inventory Changed → Shopify + Webflow



7.3 Xano Export Ready → NielsenIQ (Batch)



8. WHAT CRON STILL DOES (REDUCED ROLE)

Cron doesn't disappear – it becomes the **safety net**, not the primary driver:

TASK	BEFORE	AFTER
Customer sync	Cron every 15 min	Webhook-driven + cron catches missed webhooks
Token refresh	Cron every 15 min	Cron (no event trigger – time-based by nature)
Analytics export	Cron polls Xano	Xano trigger → webhook to Worker
SAP data sync	N/A (new)	Event-driven only
Dead letter retry	N/A (new)	Cron sweep for stuck messages
Stale event cleanup	N/A (new)	Cron expire old idempotency keys

```
// Reduced cron handler
async scheduled(_controller: ScheduledController, env: Env, _ctx: ExecutionContext): Promise<void> {
  const tenants = await listTenants(env);

  for (const shop of tenants) {
    const cfg = await resolveConfig(env, shop);

    // Token refresh – still time-based (check expiry, refresh if needed)
    await refreshShopifyTokenIfNeeded(cfg, env, shop);

    // Catch-up sync – only processes customers modified since last sync
    // that webhooks may have missed (network failures, etc.)
    await syncCustomersIncremental(cfg, env);

    // Dead letter retry – re-queue failed messages under retry limit
    await retryDeadLetterMessages(cfg, env, shop);
  }

  // Housekeeping – runs once, not per-tenant
  await cleanupExpiredIdempotencyKeys(env);
}
```

Retry strategy (per priority)

PRIORITY	MAX RETRIES	BACKOFF	DEAD LETTER AFTER
1 (critical: inventory)	5	10s, 30s, 2m, 10m, 30m	30 min
2 (high: orders)	4	30s, 2m, 10m, 1h	1 hour
3 (normal: customers)	3	2m, 15m, 1h	1 hour
4 (low: tags/segments)	3	15m, 1h, 6h	6 hours
5 (batch: analytics)	2	1h, 6h	6 hours

Dead letter queue handling

Dead letter messages are NOT discarded.

- 1. Stored in Xano: `integration_queue WHERE status = 'dead_letter'`
- 2. Archived in R2: `events/dead_letter/{date}/{event_id}.json`
- 3. Alert: POST to admin notification endpoint (email via Resend)
- 4. Manual replay: `POST /admin/events/replay { event_ids: [...] }`
- 5. Cron sweep: retry dead letters older than 1hr with exponential backoff

Circuit breaker (per destination)

```
// If a destination fails 5x in 10 minutes, stop sending for 5 minutes
interface CircuitState {
    failures: number;
    last_failure: string;
    open_until: string | null; // null = circuit closed (healthy)
}

// Stored in KV: circuit:{channel} → CircuitState
// Checked before every delivery attempt
// Auto-resets after cooldown period
```

10. OBSERVABILITY

GET /admin/events/status

```
{
  "channels": {
    "sap": { "circuit": "closed", "last_24h": { "delivered": 142, "failed": 3, "pending": 0 } },
    "nielseniq": { "circuit": "closed", "last_24h": { "delivered": 1, "failed": 0, "pending": 0 } },
    "ga4": { "circuit": "closed", "last_24h": { "delivered": 847, "failed": 0, "pending": 2 } },
  },
  "dead_letter": { "count": 3, "oldest": "2026-05-27T02:15:00Z" },
  "throughput": { "events_per_minute": 2.3, "avg_latency_ms": 1840 }
}
```

GET /admin/events/log?channel=sap&limit=20

Returns recent delivery attempts with status, latency, and error details.

Phase 1: Add ingest + deliver endpoints (non-breaking)

- New routes: `POST /events/ingest` , `POST /events/deliver`
- Existing webhook handlers unchanged
- Both patterns run in parallel

Phase 2: Redirect Shopify webhooks to unified ingest

- Update Shopify webhook addresses from `/webhooks/customer-update` to `/events/ingest?source=shopify`
- Old endpoints stay as aliases (backward compatible)

Phase 3: Add Xano task runner + DB triggers

- Create `integration_queue` table
- Create background task `process_integration_event`
- Create DB trigger on insert
- Test with R2-only delivery (archive everything, deliver nothing)

Phase 4: Enable external delivery channels

- SAP OData connector
- NielsenIQ S3 drop
- Circana API
- Each channel enabled per-tenant via config flag

Phase 5: Reduce cron to safety net

- Remove analytics polling from cron
 - Remove primary customer sync from cron (webhook-driven)
 - Keep: token refresh, dead letter sweep, catch-up sync
-

12. STAKEHOLDER ACCESS

CAPABILITY	A (CREATOR)	B (SHARED)	C (PRIVATE)
/events/ingest	✓ All sources	✓ Shopify webhooks only	✓ All sources
/events/deliver	✓ Xano callback	✗ Internal only	✓ Their Xano callback
/admin/events/status	✓	✗	✓
/admin/events/replay	✓	✗	✓
Configure destinations	✓ Platform config	✗	✓ Own config
SAP / ERP credentials	✓ Secrets	✗	✓ Own secrets
Dead letter alerts	✓ Email	✗	✓ Own alerts